

IT611 · 2026

Systems Analysis & Design

Week 1 — Foundations & the Evolving SDLC

The role of the analyst · The SDLC family of models · Where modern advances fit

Today's Agenda

1

Welcome & Course at a Glance

Objectives, structure, and what makes this course different

2

The Systems Analyst — Then & Now

How the role has evolved with modern software practice

3

The SDLC Family of Models

Waterfall, iterative, spiral, agile, and DevOps as a continuum

4

Where the Advances Fit

Platform engineering & AI-augmented development

5

Studio: Team Formation & Project Charter

Hands-on lab to kick off the semester project

The Systems Analyst: Then & Now

CLASSICAL ROLE

- Gathers requirements via structured interviews & documents
- Produces detailed up-front specifications
- Hands designs to a separate development team
- Models with formal, static UML diagrams
- Engages primarily at project start and end

MODERN ROLE

- Co-creates requirements continuously with product & users
- Designs for change: modular, service-based architectures
- Embedded across build, deploy, and operate
- Uses lightweight modeling (C4, event storming) + AI tools
- Owns quality attributes: security, scalability, observability

Textbooks move slower than the field

Most SA&D texts were written for an era of monoliths and annual releases. Today's systems are built on microservices, deployed continuously, secured by design, and increasingly assembled with AI assistance.

Cloud-native

DevSecOps

AI-assisted SDLC

Event-driven

Platform engineering

The SDLC: A Family of Models

Each generation of process models solved the failures of the one before it.



DevOps isn't a phase that ends — it's a continuum of build, deploy, operate, and learn that never stops.

The Waterfall Model



Strength: predictable, easy to plan and document.

Weakness: assumes requirements are known up front — costly to change direction once a phase closes.

Still useful for: fixed-scope contracts, regulated/compliance-heavy systems, hardware-tied releases.

Iterative & Spiral Models

ITERATIVE

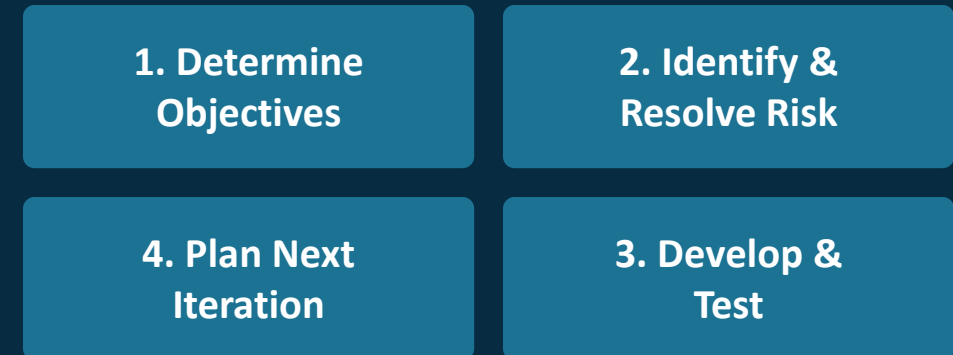
Build a small slice, learn from it, refine, repeat — each cycle adds working functionality.



↻ repeats each cycle

SPIRAL

Each loop of the spiral repeats four activities, with explicit risk analysis before committing further.

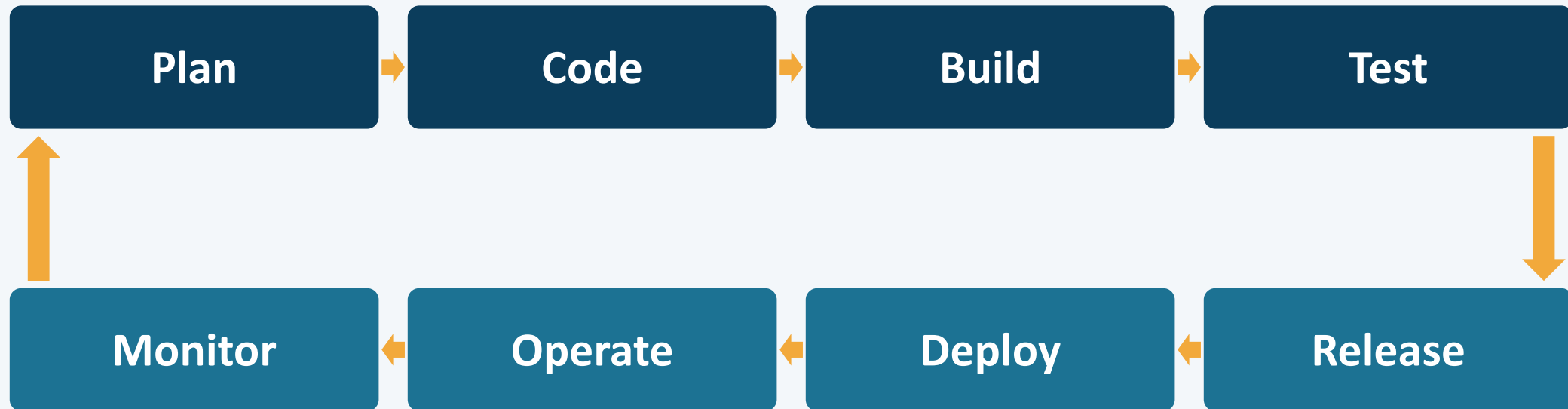


The Agile Model

Short, fixed-length iterations (sprints) deliver working software continuously, guided by customer feedback.



A Continuum, Not a Phase



DevOps SDLC blends traditional software development with IT operations into one continuous, looping process — instead of separate teams handing off work in stages, everything from planning to monitoring feeds back into itself. Automated pipelines and continuous feedback erase the line between "building" and "running" a system — analysis & design decisions now travel all the way to production.

A Continuum, Not a Phase

That loop never really stops — each turn through it feeds the next. Here's what happens at each stage:

- *Plan — Teams gather requirements, define scope, and prioritize what gets built, often using tools like Jira or Azure Boards.*
- *Code — Developers write and version the code, usually in short-lived feature branches, committing to a shared repo (Git, GitHub, GitLab).*
- *Build — The code gets compiled and packaged automatically whenever it's pushed. This is where CI (continuous integration) tools like Jenkins, GitHub Actions, or CircleCI kick in.*
- *Test — Automated test suites (unit, integration, security scans) run against the build to catch issues before they reach users.*
- *Release — Once tests pass, the build is versioned and approved for production — this is the handoff point between CI and CD (continuous delivery).*
- *Deploy — The release ships to production, often via automated pipelines using tools like ArgoCD, Spinnaker, or Kubernetes-based rollouts (blue-green, canary, etc.).*
- *Operate — The system runs live. Infrastructure-as-code (Terraform, Ansible) and container orchestration keep environments consistent and scalable.*
- *Monitor — Observability tools (Prometheus, Grafana, Datadog) track performance, errors, and user behavior — feeding insights straight back into Plan for the next cycle.*

The whole point of DevOps is collapsing the wall between "Dev" (the top half) and "Ops" (the bottom half) — instead of throwing code over a fence to operations, the same team owns it end-to-end, with automation gluing every handoff together.

DevOps Is Culture First, Tooling Second

The CALMS framework captures what separates real DevOps adoption from "we bought a CI/CD tool."



Culture

Shared ownership
across dev & ops;
blameless
postmortems, not silos



Automation

Remove manual,
error-prone steps from
build, test, and release



Lean

Small batches, fast
feedback loops,
eliminate work-in-
progress waste



Measurement

Instrument everything
— decisions follow
metrics, not opinions



Sharing

Knowledge, tooling,
and incident learnings
flow freely across
teams

The DevOps Toolchain, Stage by Stage

Plan

Jira · Azure Boards ·
Linear

Code

Git · GitHub · GitLab

Build

Maven · Gradle · npm

Test

JUnit · Selenium ·
Cypress

Release

Jenkins · GitHub
Actions · CircleCI

Deploy

Kubernetes · Helm ·
ArgoCD

Operate

Terraform · Ansible ·
Docker

Monitor

Prometheus · Grafana ·
Datadog

Example tools shown for illustration — teams choose based on cloud provider, scale, and existing stack.

Measuring DevOps Success: Four Key Metrics

Deployment Frequency

How often an organization successfully releases to production

Lead Time for Changes

Time from code commit to that code running in production

Change Failure Rate

Share of deployments that cause a failure in production

Time to Restore Service

How quickly the team recovers when an incident occurs

These four metrics — popularized by DORA (DevOps Research and Assessment) — show that speed and stability move together for high-performing teams, not against each other.

Where the Advances Fit: Platform Engineering

1

Internal Developer Platforms

Self-service infrastructure so teams ship without waiting on tickets

2

Golden Paths

Opinionated, paved-road templates for common architectures

3

Reduced Cognitive Load

Platform teams absorb complexity, so product teams focus on business logic

Design implication: analysts must design not just the system, but the paths teams take to build it.

Where the Advances Fit: AI-Augmented SDLC

1

Requirements & Review

LLMs assist elicitation, ambiguity detection, and spec review

2

Code & Test Generation

AI-assisted scaffolding, refactoring, and test-case generation

3

Design-Time Guardrails

Human review, evaluation pipelines, and traceability stay mandatory

This course treats AI as a design-time collaborator, not a replacement for analysis judgment — Week 12 goes deep on this.

What to Remember from Week 1

- ▶ The analyst's role has shifted from gatekeeper to embedded, continuous collaborator
- ▶ No single SDLC model is "correct" — fit the model to risk, scale, and change tolerance
- ▶ DevOps reframes delivery as a continuous loop, not a final phase
- ▶ Platform engineering and AI tooling are now part of the analyst's design toolkit